WEEK 2: IMPLEMENTING SECURITY MEASURES

1. Fix Vulnerabilities

- **Sanitize and Validate Inputs**:
- **Password Hashing**

In index.js we are follow this code for password hashing and user validation

```
const validator = require('validator');
const bcrypt = require('bcrypt');
app.post('/login', async function(req, res) {
  const { username, password } = req.body;

  // Validate username (not empty)
  if (!username || !password) {
    return res.status(400).send("Missing credentials");
  }
  pool.query('SELECT * FROM admin WHERE username = ?', [username], async function(err, rows) {
    if (err || rows.length === 0) {
      return res.status(400).send("User not found");
    }

    const isMatch = await bcrypt.compare(password, rows[0].password);

    if (isMatch) {
      // Create JWT token (with a payload and secret key)
      const token = jwt.sign({ id: rows[0].id, username: rows[0].username }, 'your-secret-key', { expiresIn: '1h' });
```

```
            console.log(token)
            // Send token back to the client
            res.status(200).send({
                message: "Authentication successful",
                token: token
            });
        } else {
            return res.status(401).send("Invalid credentials");
        }
    });
});
```

2. Enhance Authentication

Add middleware for token Authentication and Add this in all curd operations

Middleware/ authenticateToken.js:

```
const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1]; // "Bearer <token>"

    if (!token) return res.status(401).send('Access Denied');

    jwt.verify(token, 'your-secret-key', (err, user) => {
        if (err) return res.status(403).send('Invalid Token');
        req.user = user;
        next();
```

```
    });
}
module.exports = authenticateToken;
```

index.js:

```
const jwt = require('jsonwebtoken');
const authenticateToken = require('./middleware/authenticateToken');
const isMatch = await bcrypt.compare(password, rows[0].password);


        if (isMatch) {
            // Create JWT token (with a payload and secret key)
            const token = jwt.sign({ id: rows[0].id, username: rows[0].username }, 'your-secret-key', { expiresIn: '1h' });
            console.log(token)
            // Send token back to the client
            res.status(200).send({
                message: "Authentication successful",
                token: token
            });
        } else {
            return res.status(401).send("Invalid credentials");
        }
pp.post('/create', authenticateToken, function(req, res) {
    console.log("In Create Post");
    console.log("Req Body : ", req.body);
    var userData = { "name": req.body.Name, "studentID": req.body.StudentID, "department": req.body.Department };
```

```
    pool.query('INSERT INTO user SET ?', userData, function (err) {
        if (err) {
            console.log("unable to insert into database");
            return res.status(400).send("unable to insert into database");
        } else {
            console.log("User Added Successfully!!!!");
            res.status(200).send("User Added");
        }
    });
});

app.get('/list', authenticateToken, function(req, res) {
    console.log("Inside list users");
    pool.query('SELECT * FROM user', (err, result) => {
        if (err) {
            console.log(err);
            return res.status(400).send("Error in Connection");
        } else {
            console.log("users list: ", JSON.stringify(result));
            res.status(200).send(result);
        }
    });
});

app.delete('/delete/:id', authenticateToken, function(req, res) {
    console.log("In Delete Post");
```

```
  console.log("The user to be deleted is ", req.params.id);


  pool.query('DELETE FROM user WHERE studentID = ?', [req.params.id], (err, rows) => {
    if (err) {
      console.log("User Not Found");
      return res.status(400).send("User not found");
    } else {
      console.log("User ID " + req.params.id + " was removed successfully");
      pool.query('SELECT * FROM user', (err, result) => {
        if (err) return res.status(400).send("Error in Connection");
        res.status(200).send(result);
      });
    }
  });
});
```

3. Secure Data Transmission

We add helmet headers help prevent common web vulnerabilities like cross-site scripting (XSS), clickjacking, and sniffing attack.

INDEX.js

```
const helmet = require('helmet');
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'", "'unsafe-inline'"], // update based on frontend needs
    objectSrc: ["'none'"],
    upgradeInsecureRequests: [],
```

```
  }
}));
```